

H1 - Smartphonependel

Fynn Kanja 3780065, Valentino D'Agosto 3763229

Invalid Date

Inhaltsverzeichnis

1	1. Motivation	3
2	1.1 Aufgabenstellung	4
3	2. Aufgabe 1: Messung der Beschleunigungs- und Gyroskopdaten	5
4	3. Bestimmung der Pendelfrequenz	9
4.1	3.1 Auswertung der linearen Beschleunigung	9
4.2	3.2 Bestimmung der Pendelfrequenz mittels Curve Fitting	12
4.3	3.3 Aus den Gyroskopdaten ermitteln	15
4.4	3.5 Berechnung der Pendelfrequenz	19
4.5	4. Bestimmung der Dämpfungskonstante	20
5	5. Bestimmung der Komponenten der Beschleunigung	23
6	6. Fehlerbetrachtungen	31
7	6. Fehlerbetrachtung	32
8	7. Quellen	33

1 1. Motivation

Ziel dieses Versuchs ist es, die Frequenz, Dämpfungskonstante sowie die Beschleunigungskomponenten des selber gebauten Smartphone-Pendels experimentell zu bestimmen. Des Weiteren werden mithilfe der theoretischen Grundlagen Erwartungswerte berechnet und diese mit den experimentell bestimmten Werten verglichen. Das Experiment selber wurde mehrfach durchgeführt und die Werte des am besten gelungenen Versuchs werden für das Protokoll genutzt.

2 1.1 Aufgabenstellung

1. Verwenden Sie Ihr Smartphone als Pendelkörper und messen Sie simultan die Winkelgeschwindigkeit (Drehrate), die Beschleunigung, und die lineare Beschleunigung (Beschleunigung ohne g) ihres Smartphones während des Pendelvorgangs.
2. Bestimmen Sie die Pendelfrequenz mit drei unterschiedlichen Methoden. Berechnen Sie den theoretischen Wert. Liegt dieser noch innerhalb der Messabweichung?
3. Bestimmen Sie die Dämpfungskonstante.
4. Berechnen Sie die Komponenten der Beschleunigung unter Verwendung der gemessenen Drehrate und vergleichen Sie mit der gemessenen Beschleunigungswerten (mit und ohne g). Stellen Sie dazu die gemessenen und die berechneten Beschleunigungskomponenten in jeweils einem Diagramm dar.

3 2. Aufgabe 1: Messung der Beschleunigungs- und Gyroskopdaten

Für die Messung der Daten wurde ein Handy mit der Phyphox App genutzt. Mithilfe der App kann man die Rohdaten aus den verschiedensten Sensoren auslesen, die in einem Smartphone verbaut sind. Während des Experiments wurden Beschleunigung, lineare Beschleunigung mit dem Beschleunigungssensor und Winkelgeschwindigkeit mithilfe des Gyroskopsensors auf allen drei Achsen gleichzeitig gemessen.

Der Aufbau des Experiments bestand, wie in Abbildung 2 zu sehen, aus einem Smartphone, welches an zwei parallelen Fäden, wie bei einer Schaukel, um die Rotation während des Pendels zu minimieren, an einem Regenschirm befestigt wurde. Dabei zeigte die Handyoberfläche parallel zum Boden den Einfluss des Luftwiderstandes zu minimieren. Somit befindet sich die positive z-Achse in Richtung Boden, die positive y-Achse zeigt nach links und die positive x-Achse zur Richtung der Kamera. Die gesamte Pendellänge mit Handyhülle und Fadenlänge betrug 93 cm, wobei die Fäden 92 cm lang waren.

Bei Beginn der Messung befand sich das System in der Ruhelage und wurde erst danach ausgelenkt, damit das Gyroskop kalibriert war und den Wert 0 in der Ruhelage anzeigt. Beim Auslenken wurde nur um einen kleinen Winkel ausgelenkt, damit man das Pendel vereinfacht als mathematisches Pendel betrachten kann, da die Kleinwinkelnäherung angewendet werden kann. Beim Loslassen wurde versucht, dem Smartphone möglichst wenig Drehung und Schwung mitzugeben, sodass sich das Handy beim Pendeln nicht dreht oder die Messwerte verfälscht. Zur Minimierung möglicher Messfehler wurde das Pendel zunächst für 20 Sekunden eingeschwungen, um möglichst reine Messdaten zu erhalten.

Während des insgesamt 143 Sekunden andauernden Versuchs wurden Messdaten aufgezeichnet, die anschließend analysiert wurden, die in der folgenden Abbildung 1 präsentiert wrden.

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# Gemeinsame Einstellungen
end_time = 120
start_time = 20.3
```

```

# 3x2 Subplot erstellen
fig, axs = plt.subplots(3, 2, figsize=(12, 8))
fig.suptitle('Sensordaten Übersicht', fontsize=18)

# Beschleunigungsdaten (Spalte 1)
df_acc = pd.read_csv('data/Accelerometer.csv')
df_acc['Zeit_Angepasst'] = df_acc['Time (s)'] - start_time

axs[0, 0].plot(df_acc['Zeit_Angepasst'], df_acc['Acceleration x (m/s^2)'], color='r')
axs[0, 0].set_title('Beschleunigung X')
axs[0, 0].set_ylabel('Beschleunigung (m/s²)')
axs[0, 0].set_xlim(0, end_time)
axs[0, 0].set_ylim(0, 2)
axs[0, 0].grid(True)

axs[1, 0].plot(df_acc['Zeit_Angepasst'], df_acc['Acceleration y (m/s^2)'], color='r')
axs[1, 0].set_title('Beschleunigung Y')
axs[1, 0].set_ylabel('Beschleunigung (m/s²)')
axs[1, 0].set_xlim(0, end_time)
axs[1, 0].set_ylim(-0.3, 0.4)
axs[1, 0].grid(True)

axs[2, 0].plot(df_acc['Zeit_Angepasst'], df_acc['Acceleration z (m/s^2)'], color='r')
axs[2, 0].set_title('Beschleunigung Z')
axs[2, 0].set_ylabel('Beschleunigung (m/s²)')
axs[2, 0].set_xlabel('Zeit (s)')
axs[2, 0].set_xlim(0, end_time)
axs[2, 0].set_ylim(-11, -8.5)
axs[2, 0].grid(True)

# Gyroskop-Daten (Spalte 2)
df_gyro = pd.read_csv('data/Gyroscope.csv')
df_gyro['Zeit_Angepasst'] = df_gyro['Time (s)'] - start_time

axs[0, 1].plot(df_gyro['Zeit_Angepasst'], df_gyro['Gyroscope x (rad/s)'], color='g')
axs[0, 1].set_title('Gyroskop X')
axs[0, 1].set_ylabel('Drehrate (rad/s)')
axs[0, 1].set_xlim(0, end_time)
axs[0, 1].set_ylim(-0.25, 0.25)
axs[0, 1].grid(True)

axs[1, 1].plot(df_gyro['Zeit_Angepasst'], df_gyro['Gyroscope y (rad/s)'], color='g')

```

```

axs[1, 1].set_title('Gyroskop Y')
axs[1, 1].set_ylabel('Drehrate (rad/s)')
axs[1, 1].set_xlim(0, end_time)
axs[1, 1].set_ylim(-1.11, 1.11)
axs[1, 1].grid(True)

axs[2, 1].plot(df_gyro['Zeit_Angepasst'], df_gyro['Gyroscope z (rad/s)'], color='g')
axs[2, 1].set_title('Gyroskop Z')
axs[2, 1].set_ylabel('Drehrate (rad/s)')
axs[2, 1].set_xlabel('Zeit (s)')
axs[2, 1].set_xlim(0, end_time)
axs[2, 1].set_ylim(-1.5, 1.5)
axs[2, 1].grid(True)

# Lineare Beschleunigungsdaten (Spalte 3)
#df_lin = pd.read_csv('data/Linear Acceleration.csv')
#df_lin['Zeit_Angepasst'] = df_lin['Time (s)'] - start_time

#axs[0, 2].plot(df_lin['Zeit_Angepasst'], df_lin['Linear Acceleration x (m/s^2)'], color='b')
#axs[0, 2].set_title('Lineare Beschleunigung X')
#axs[0, 2].set_ylabel('Lin. Beschl. (m/s^2)')
#axs[0, 2].set_xlim(0, end_time)
#axs[0, 2].set_ylim(-2.8, 2.8)
#axs[0, 2].grid(True)

#axs[1, 2].plot(df_lin['Zeit_Angepasst'], df_lin['Linear Acceleration y (m/s^2)'], color='b')
#axs[1, 2].set_title('Lineare Beschleunigung Y')
#axs[1, 2].set_ylabel('Lin. Beschl. (m/s^2)')
#axs[1, 2].set_xlim(0, end_time)
#axs[1, 2].set_ylim(-2, 2)
#axs[1, 2].grid(True)

#axs[2, 2].plot(df_lin['Zeit_Angepasst'], df_lin['Linear Acceleration z (m/s^2)'], color='b')
#axs[2, 2].set_title('Lineare Beschleunigung Z')
#axs[2, 2].set_ylabel('Lin. Beschl. (m/s^2)')
#axs[2, 2].set_xlabel('Zeit (s)')
#axs[2, 2].set_xlim(0, end_time)
#axs[2, 2].set_ylim(-1.5, 1.5)
#axs[2, 2].grid(True)

# Legende für die gesamten Spalten erstellen
handles = [

```

```

plt.Line2D([0], [0], color='r', label='Beschleunigung'),
plt.Line2D([0], [0], color='g', label='Drehrate'),
#plt.Line2D([0], [0], color='b', label='Lineare Beschleunigung')
]
fig.legend(handles=handles, loc='lower center', ncol=3, bbox_to_anchor=(0.5, 0.01))

plt.tight_layout(rect=[0, 0.03, 1, 0.97]) # Anpassung für die Legende unten
plt.subplots_adjust(hspace=0.3, wspace=0.3) # Abstand zwischen den Subplots erhöhen
plt.show()

```

Sensordaten Übersicht

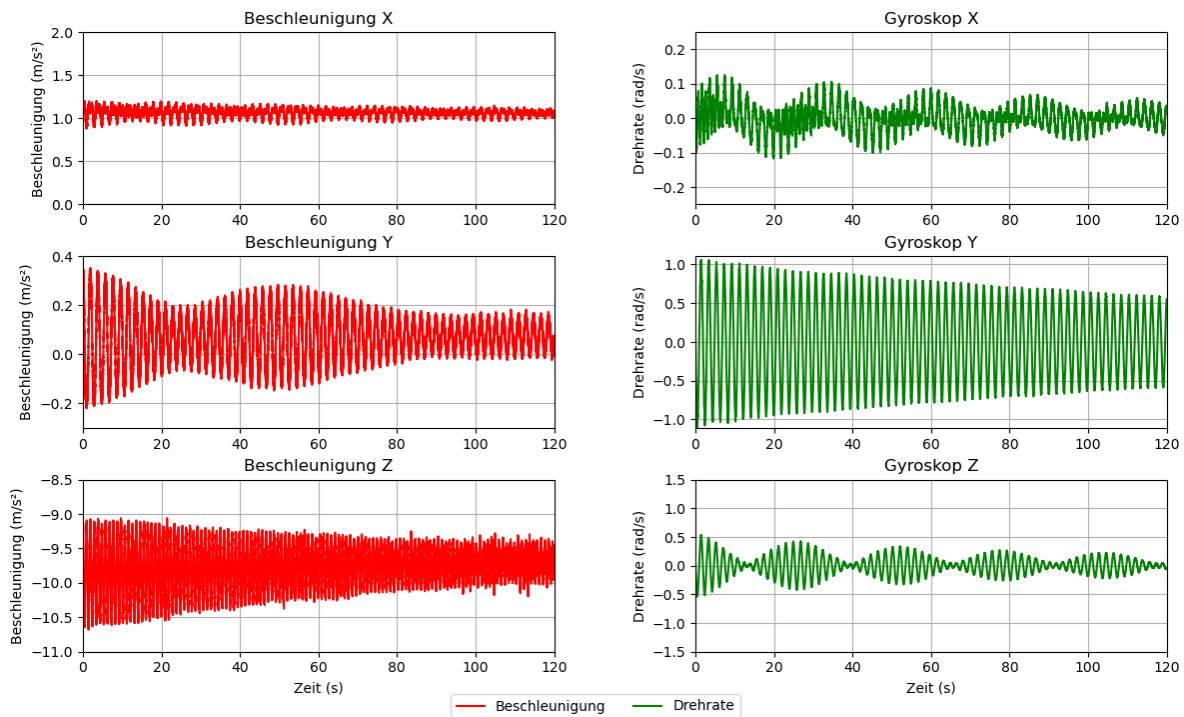


Abbildung 1: Übersicht der gemessenen Daten

Abbildung 2: Aufbau des Pendelversuchs

4 3. Bestimmung der Pendelfrequenz

4.1 3.1 Auswertung der linearen Beschleunigung

Um die Frequenz des Pendels zu bestimmen, analysieren wir die Zeitpunkte der Beschleunigungsmaxima, die im obersten Diagramm hervorgehoben sind. Die ermittelten Spitzen treten bei den Zeitpunkten $t = 0.8\text{s}$, $t = 2.73\text{s}$, $t = 4.7\text{s}$, $t = 6.64\text{s}$ und $t = 8.6\text{s}$ auf.

Die Periodendauer T zwischen zwei aufeinanderfolgenden Peaks lässt sich durch die Differenz der entsprechenden Zeitpunkte berechnen:

$$T = t_{i+1} - t_i$$

Die Frequenz f des Pendels ergibt sich schließlich als Kehrwert der Periodendauer:

$$f = \frac{1}{T}$$

Periode	Zeit (s)	Periodendauer (s)	Frequenz (1/s)
0	0,8	–	–
1	2,75	1,95	0,513
2	4,7	1,95	0,513
3	6,64	1,94	0,515
4	8,6	1,96	0,510
Durchschnitt	–	1,95	0,513

Tabelle 1: Übersicht der Daten

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.signal import find_peaks

end_time = 10
```

```

# Read Linear Acceleration.csv file
df = pd.read_csv('data/Linear Acceleration.csv')

# Erstelle neue Zeitachse, die bei 0 beginnt (durch Subtraktion des Startwertes)
start_time = 20.3
df['Zeit_Angepasst'] = df['Time (s)'] - start_time

# Plot mit der angepassten Zeitachse
fig, axs = plt.subplots(3, 1, figsize=(10, 8))
fig.suptitle('Lineare Beschleunigung', fontsize=16)

axs[0].plot(df['Zeit_Angepasst'], df['Linear Acceleration x (m/s^2)'], label='Lineare Beschleunigung x (m/s^2)')
axs[0].set_ylabel('Lineare Beschleunigung x (m/s^2)')
axs[0].set_xlabel('Zeit (s)')
axs[0].set_xlim(0, end_time) # Entspricht 20.3 bis 50.3 in den Originaldaten
axs[0].set_ylim(-2.8, 2.8)
axs[0].grid()
axs[0].xaxis.set_major_locator(plt.MultipleLocator(0.5))
axs[0].xaxis.set_minor_locator(plt.MultipleLocator(0.1))
axs[0].plot([0.8, 0.8], [4,-4], color='b' , linestyle='--')
axs[0].plot([2.75,2.75], [4,-4], color='b' , linestyle='--')
axs[0].plot([4.7,4.7], [4,-4], color='b' , linestyle='--')
axs[0].plot([6.64,6.64], [4,-4], color='b' , linestyle='--')
axs[0].plot([8.6,8.6], [4,-4], color='b' , linestyle='--')

axs[1].plot(df['Zeit_Angepasst'], df['Linear Acceleration y (m/s^2)'], label='Lineare Beschleunigung y (m/s^2)')
axs[1].set_ylabel('Lineare Beschleunigung y (m/s^2)')
axs[1].set_xlabel('Zeit (s)')
axs[1].set_xlim(0, end_time) # Entspricht 20.3 bis 50.3 in den Originaldaten
axs[1].xaxis.set_major_locator(plt.MultipleLocator(0.5))
axs[1].xaxis.set_minor_locator(plt.MultipleLocator(0.1))
axs[1].set_ylim(-2, 2)
axs[1].grid()

axs[2].plot(df['Zeit_Angepasst'], df['Linear Acceleration z (m/s^2)'], label='Lineare Beschleunigung z (m/s^2)')
axs[2].set_ylabel('Lineare Beschleunigung z (m/s^2)')
axs[2].set_xlabel('Zeit (s)')
axs[2].set_xlim(0, end_time) # Entspricht 20.3 bis 50.3 in den Originaldaten
axs[2].xaxis.set_major_locator(plt.MultipleLocator(0.5))
axs[2].xaxis.set_minor_locator(plt.MultipleLocator(0.1))
axs[2].set_ylim(-1.5, 1.5)
axs[2].grid()

```

```
# Berechne die Winkelbeschleunigung
df['Winkelbeschleunigung x (rad/s^2)'] = np.gradient(df['Linear Acceleration x (m/s^2)'], df
df['Winkelbeschleunigung y (rad/s^2)'] = np.gradient(df['Linear Acceleration y (m/s^2)'], df
df['Winkelbeschleunigung z (rad/s^2)'] = np.gradient(df['Linear Acceleration z (m/s^2)'], df

plt.tight_layout()
plt.show()
```

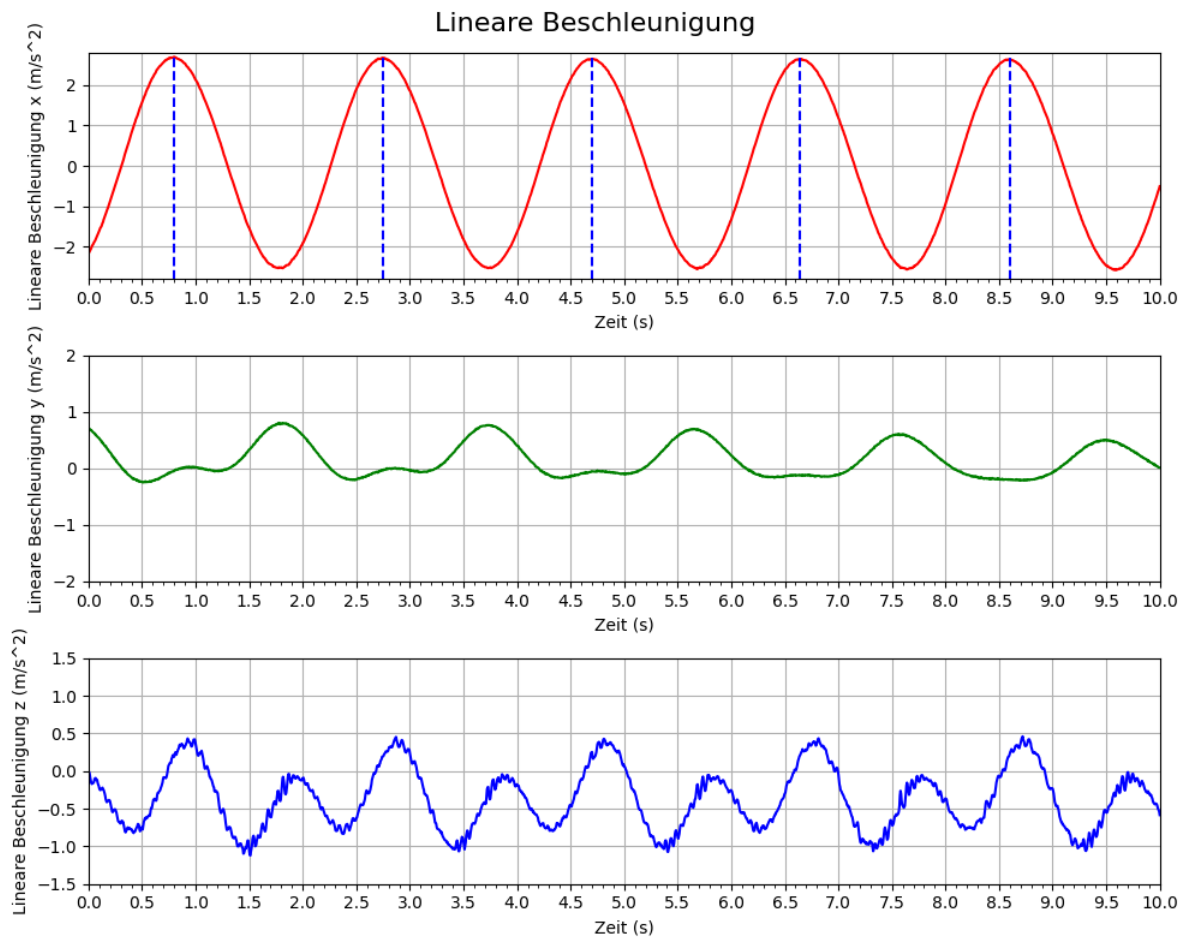


Abbildung 3: Übersicht der gemessenen Daten der linearen Beschleunigung

4.2 3.2 Bestimmung der Pendelfrequenz mittels Curve Fitting

Um die Frequenz des Pendels zu bestimmen, wurde eine Sinusfunktion an die Messdaten der Beschleunigung angepasst. Diese Methode eignet sich besonders gut, da das Signal durch periodische, schwingende Bewegungen geprägt ist – ein typisches Verhalten bei einem Pendel. Der Vorteil des Curve-Fittings gegenüber Methoden wie der Fourier-Analyse liegt darin, dass die Frequenz direkt als Parameter einer Sinusfunktion geschätzt wird und zusätzlich Informationen über Amplitude, Phase und Offset bereitgestellt werden.

Für die Auswertung wurde die y-Achse der Beschleunigungsdaten gewählt. Diese zeigt eine klare periodische Struktur mit einer gut ausgeprägten Sinusform, die für ein stabiles und zuverlässiges Fit-Ergebnis erforderlich ist. Im Vergleich dazu ist die x-Achse stark verrauscht und die z-Achse weist zwar Schwingungen auf, ist jedoch durch die konstante Gravitation stark negativ überlagert. Zudem war die y-Achse auch diejenige, auf der das Smartphone geschwungen ist.

Die Anpassung der Kurve an die y-Komponente ergab eine Frequenz von: $f_{freq} = 0,513$ Hz, wie in der folgenden Abbildung als gestrichelte Linie im zweiten Diagramm dargestellt ist

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import scipy.optimize

end_time = 20

# Read Accelerometer.csv file
df = pd.read_csv('data/Accelerometer.csv')

# Erstelle neue Zeitachse, die bei 0 beginnt (durch Subtraktion des Startwertes)
start_time = 20.3
df['Zeit_Angepasst'] = df['Time (s)'] - start_time

# Plot mit der angepassten Zeitachse
fig, axs = plt.subplots(3, 1, figsize=(10, 8))
fig.suptitle('Beschleunigungsdaten', fontsize=16)

axs[0].plot(df['Zeit_Angepasst'], df['Acceleration x (m/s^2)'], label='Beschleunigung x (m/s^2)')
axs[0].set_ylabel('Beschleunigung x (m/s^2)')
axs[0].set_xlabel('Zeit (s)')
```

```

axs[0].set_xlim(0, end_time) # Entspricht 20.3 bis 50.3 in den Originaldaten
axs[0].set_ylim(0, 2)
axs[0].grid()

axs[1].plot(df['Zeit_Angepasst'], df['Acceleration y (m/s^2)'], label='Beschleunigung y (m/s^2)')
axs[1].set_ylabel('Beschleunigung y (m/s^2)')
axs[1].set_xlabel('Zeit (s)')
axs[1].set_xlim(0, end_time)
axs[1].set_ylim(-0.4, 0.4)
axs[1].grid()

def sinfunc(t, A, w, p, g, _lambda, c2):
    return A * np.exp(-_lambda*t) * np.sin(np.exp(-g*t)*w*t + p) + c2

# Initial parameters based on data analysis
# Somehow the curve will fit better if we add another damping constant
p0 = [
    0.4,          # A: amplitude
    2*np.pi*0.5, # w: starting frequency
    0,            # p: phase shift
    0.002,        # g: damping constant for frequency
    0.04,         # lambda: damping constant for amplitude
    0,            # c2: vertical offset
]

mask = df['Zeit_Angepasst'] <= end_time
t_fit = df['Zeit_Angepasst'][mask]
y_fit = df['Acceleration y (m/s^2)'][mask] + 0.04

popt, pcov = scipy.optimize.curve_fit(sinfunc, t_fit, y_fit, p0=p0)
A, w, p, g, _lambda, c2 = popopt
f = w/(2.*np.pi)
print(f"Erhaltene Frequenz: {f:.3f} Hz")
#print(f"Dämpfungskonstante: {g}") => -0.0011682543226750118
#print(f"Dämpfungskonstante: {_lambda}") => 0.01491478347667464

t_smooth = np.linspace(0, end_time, 1000)
fit_smooth = sinfunc(t_smooth, *popt)
axs[1].plot(t_smooth, fit_smooth, 'r--', label=f'Fit (f={f:.3f} Hz)')
axs[1].legend(loc='upper right')

```

```

axs[2].plot(df['Zeit_Angepasst'], df['Acceleration z (m/s^2)'], label='Beschleunigung z (m/s^2)')
axs[2].set_ylabel('Beschleunigung z (m/s^2)')
axs[2].set_xlabel('Zeit (s)')
axs[2].set_xlim(0, end_time) # Entspricht 20.3 bis 50.3 in den Originaldaten
axs[2].set_ylim(-11, -8.5)
axs[2].grid()
axs[1].xaxis.set_major_locator(plt.MultipleLocator(0.5))
axs[1].xaxis.set_minor_locator(plt.MultipleLocator(0.1))

plt.tight_layout()
plt.show()

```

Fitted frequency: 0.513 Hz

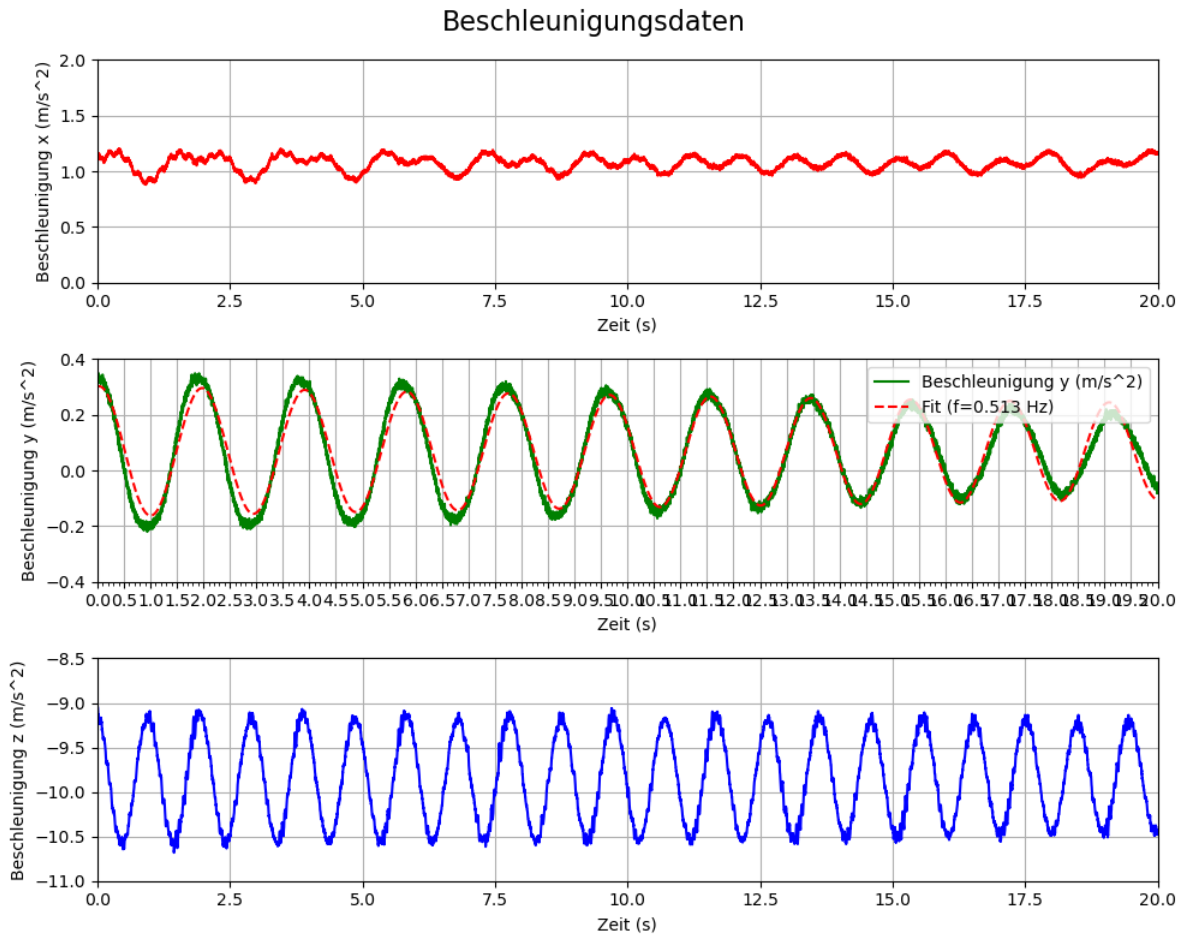


Abbildung 4: Übersicht der gemessenen Beschleunigung mit curve-fit zum bestimmen der Frequenz

4.3 3.3 Aus den Gyroskopdaten ermitteln

Zur Bestimmung der Pendelfrequenz ist zunächst die relevante Komponente der Gyroskopdaten zu identifizieren. Aufgrund des Versuchsaufbaus, bei dem sich das Smartphone während der Schwingung primär um die y -Achse dreht, ist die Gyroskop-Komponente in y -Richtung maßgeblich.

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

```

end_time = 120

# Read Gyroscope.csv file
df = pd.read_csv('data/Gyroscope.csv')

# Erstelle neue Zeitachse, die bei 0 beginnt (durch Subtraktion des Startwertes)
start_time = 20.3
df['Zeit_Angepasst'] = df['Time (s)'] - start_time

# Plot mit der angepassten Zeitachse
fig, axs = plt.subplots(3, 1, figsize=(10, 8))
fig.suptitle('Gyroskopdaten', fontsize=16)

axs[0].plot(df['Zeit_Angepasst'], df['Gyroscope x (rad/s)'], label='Gyroskop x (rad/s)', col
axs[0].set_ylabel('Gyroskop x (rad/s)')
axs[0].set_xlabel('Zeit (s)')
axs[0].set_xlim(0, end_time) # Entspricht 20.3 bis 50.3 in den Originaldaten
axs[0].set_ylim(-0.25, 0.25)
axs[0].grid()

axs[1].plot(df['Zeit_Angepasst'], df['Gyroscope y (rad/s)'], label='Gyroskop y (rad/s)', col
axs[1].set_ylabel('Gyroskop y (rad/s)')
axs[1].set_xlabel('Zeit (s)')
axs[1].set_xlim(0, end_time) # Entspricht 20.3 bis 50.3 in den Originaldaten
axs[1].set_ylim(-1.11, 1.11)
axs[1].grid()

axs[2].plot(df['Zeit_Angepasst'], df['Gyroscope z (rad/s)'], label='Gyroskop z (rad/s)', col
axs[2].set_ylabel('Gyroskop z (rad/s)')
axs[2].set_xlabel('Zeit (s)')
axs[2].set_xlim(0, end_time) # Entspricht 20.3 bis 50.3 in den Originaldaten
axs[2].set_ylim(-1.5, 1.5)
axs[2].grid()

plt.tight_layout()
plt.show()

```

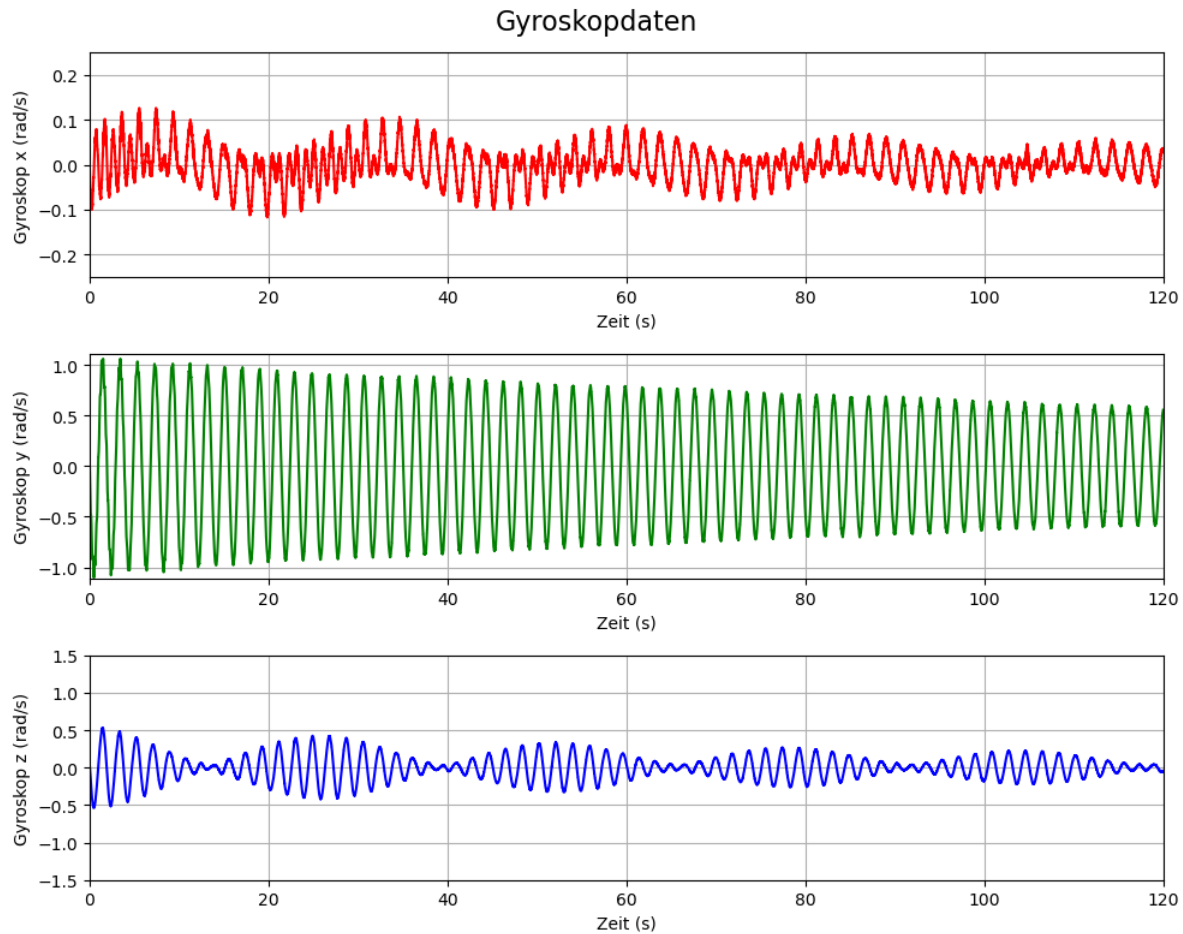



Abbildung 5: Übersicht der Gyroskopdaten Daten

Aus dem obigen Zeitdiagramm allein lässt sich die dominante Frequenz der sinusförmigen Bewegung jedoch nur schwer bestimmen. Daher wird zur Frequenzanalyse eine Fourier-Transformation durchgeführt.

Wie in Abbildung 6 ersichtlich, ergibt die Frequenzanalyse mittels Fourier-Transformation eine dominante Pendelfrequenz von $f_{gyro} = 0,516$ Hz.

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.fft import fft, fftfreq
from scipy.signal import find_peaks

# Read Gyroscope.csv file
df = pd.read_csv('data/Gyroscope.csv')
```

```

# Erstelle neue Zeitachse, die bei 0 beginnt
start_time = 20.3
df['Zeit_Angepasst'] = df['Time (s)'] - start_time

# Konvertiere die pandas Series zu NumPy-Arrays für die FFT
gyro_data = df['Gyroscope y (rad/s)'].to_numpy()

# Berechnung der Fourier-Transformation
N = len(gyro_data)
T = df['Zeit_Angepasst'].iloc[1] - df['Zeit_Angepasst'].iloc[0] # Zeitintervall
yf = fft(gyro_data) # Verwende NumPy-Array statt pandas Series
xf = fftfreq(N, T)[:N//2]

# Plot der Fourier-Transformation
fig, ax = plt.subplots(figsize=(10, 6))
amplitudes = 2.0/N * np.abs(yf[0:N//2])
ax.plot(xf, amplitudes, color='g')
ax.set_title('Fourier-Transformation von Gyroskop y')
ax.set_xlabel('Frequenz (Hz)')
ax.set_ylabel('Amplitude')
ax.set_xlim(0, 5) # Frequenzen bis 5 Hz anzeigen
ax.set_ylim(0, 1.5)
ax.grid()

# Finden von Peaks in der Fourier-Transformation
peaks, _ = find_peaks(amplitudes, height=0.1)
ax.plot(xf[peaks], amplitudes[peaks], "x", color='r', markersize=10, label='Peaks')
ax.legend()
ax.set_ylim(0, 1)
#set x axis steps to 0.5
ax.xaxis.set_major_locator(plt.MultipleLocator(0.5))
ax.xaxis.set_minor_locator(plt.MultipleLocator(0.1))

# Annotieren der Peaks mit Frequenzen
for i, peak in enumerate(peaks):
    freq = xf[peak]
    amp = amplitudes[peak]
    # Vermeide Überlappungen bei der Textplatzierung
    y_offset = 0.3 + (i % 3) * 0.15 # Versetzte Positionen für bessere Lesbarkeit

    ax.annotate(f'Peak {i+1}: {freq:.3f} Hz',
                xy=(freq, amp),

```

```

xytext=(freq, amp + y_offset),
arrowprops=dict(facecolor='black', shrink=0.05, width=1),
fontsize=12,
color='r',
ha='center')

plt.tight_layout()
plt.show()

```

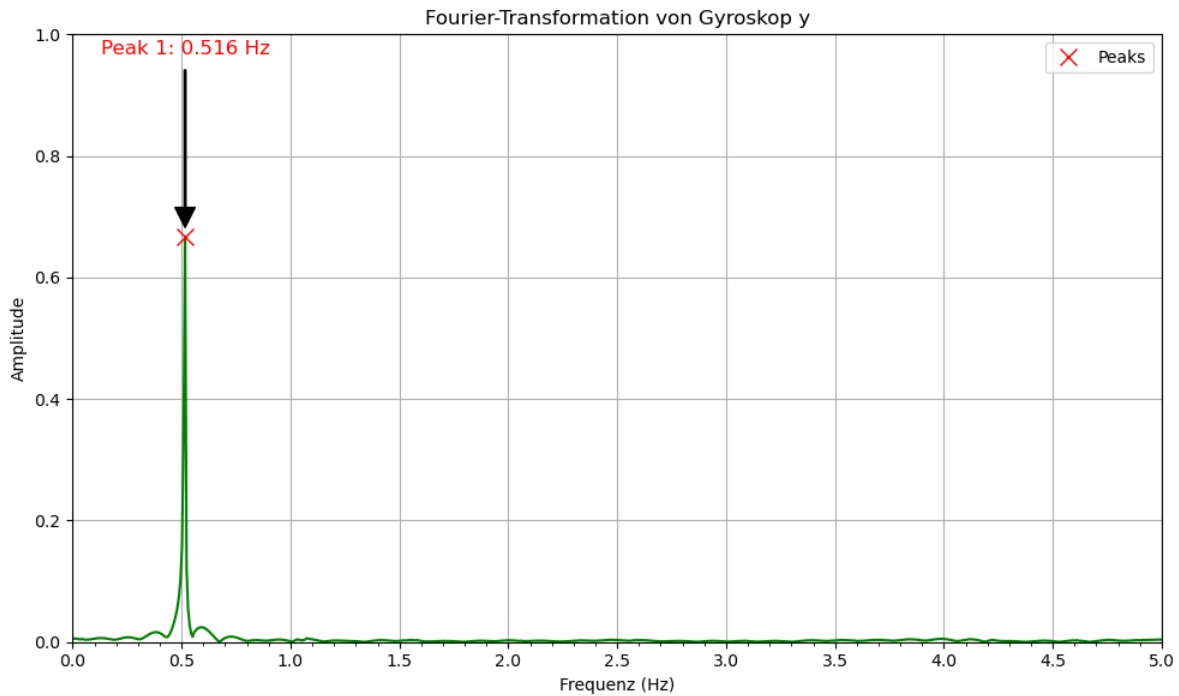


Abbildung 6: Fourieranalyse der Gyroskopdaten

4.4 3.5 Berechnung der Pendelfrequenz

Für ein einfaches mathematisches Pendel, wo Fadenmasse, Reibung sowie Luftwiderstand vernachlässigt werden können, gilt

$$T = 2\pi \cdot \sqrt{\frac{l}{g}}$$

Diese Formel lässt sich mit $f = \frac{1}{T}$ einfach nach der Frequenz umstellen

$$f = \frac{1}{2\pi} \cdot \sqrt{\frac{g}{l}}$$

wobei g als $9,81 \frac{m}{s^2}$ angenommen wird. Mit der gemessenen Fadenlänge $l = 0,93m$ ergibt sich somit eine theoretische Frequenz von $f = 0,5169$ Hz, was den aus den Daten ermittelten Frequenzen sehr nahekommt. Von den Messabweichungen liegen alle Frequenzen unter $\pm 0,01$ Hz, was als sehr genau betrachtet werden kann.

4.5 4. Bestimmung der Dämpfungskonstante

Um die Dämpfungskonstante zu bestimmen, mussten wir uns erstmal die Schwinung genauer betrachten. So wie das Pendel aufgebaut wurde, sieht man die Schwinung am besten an der y-Achse des Smartphones, weshalb wir die Gyroskopdaten genau dieser Achse nutzen, um die Dämpfung zu bestimmen. In der Theorie übt sich die Dämpfung auf die eigentliche Schwingung als exponentieller Abfall aus. Wenn wir unsere eigentliche Schwingung als $\varphi(t) = \varphi_0 \cdot \cos(\omega_0 t)$ bezeichnen und unsere Dämpfungskonstante δ ist, bildet sich die gedämpfte Schwingung:

$$\varphi(t) = \varphi_0 \cdot e^{-\delta t} \cdot \cos(\omega_0 t)$$

Um δ daraus zu extrahieren, beobachten wir den Betrag der Amplituden und deren zeitliches Abnehmen. Die erste Amplitude sei nun A_0 und die anderen folgen dann aus $A(t) = A_0 \cdot e^{-\delta t}$. Wenn wir den Logarithmus anwenden, bildet sich $\ln A(t) = \ln A_0 - \delta t$ und damit kann man aus einer Fit-Funktion der logarithmierten Amplituden die Dämpfungskonstante bestimmen. Es wird die Funktion der logarithmierten Amplituden als $f(x)$ bezeichnet und mit das ablesen von δ geht durch $f(x) = \ln A_0 - \delta \cdot t$.

Mithilfe dieses Verfahrens wurde auch in diesem Fall gearbeitet. Mithilfe der Scipy Funktion "find_peaks" werden die Amplituden gefunden und bei der Fit-Funktion kann δ sehr leicht bestimmt werden. Diese wird mit den Plots der Amplituden und Fit-Funktion ausgegeben.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.signal import find_peaks
from scipy.optimize import curve_fit

start_time = 20.3
end_time = 100.3

# CSV-Datei laden und Zeitfenster anwenden
df = pd.read_csv("data/Gyroscope.csv")
```

```

df = df[(df['Time (s)'] >= start_time) & (df['Time (s)'] <= end_time)]
df['Zeit_Angepasst'] = df['Time (s)'] - start_time

t = df['Zeit_Angepasst']
y = df['Gyroscope y (rad/s)']

# Peaks finden
peaks, _ = find_peaks(y, distance=30)
t_peaks = t.iloc[peaks]
y_peaks = y.iloc[peaks]

# Logarithmus der Amplituden
log_amplituden = np.log(np.abs(y_peaks))

# Fit-Funktion
def linear(t, a, delta):
    return a - delta * t

# Curve-Fit anwenden
params, _ = curve_fit(linear, t_peaks, log_amplituden)
a_fit, delta = params

print(f"Dämpfungskonstante delta: {delta:.4f} 1/s")

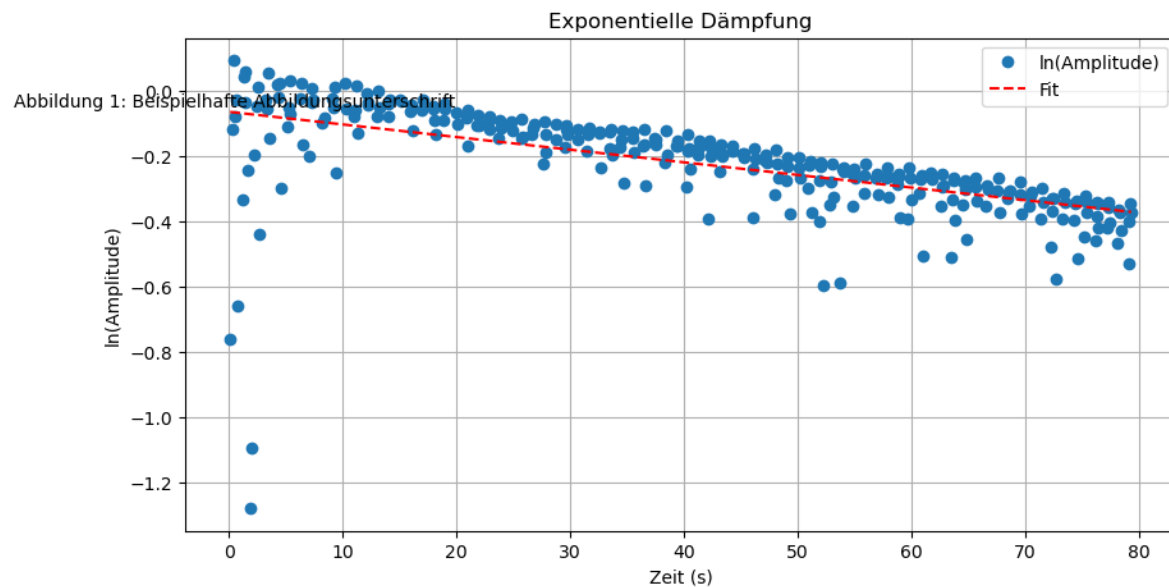
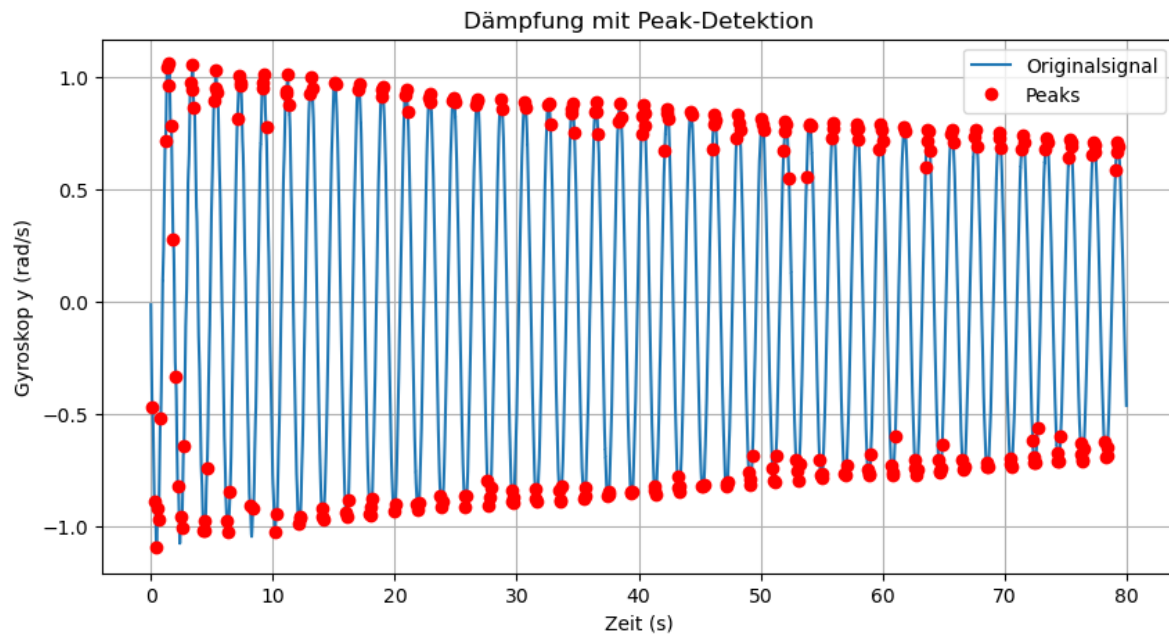
# Plot: Originalsignal mit Peaks
plt.figure(figsize=(10, 5))
plt.plot(t, y, label="Originalsignal")
plt.plot(t_peaks, y_peaks, "ro", label="Peaks")
plt.xlabel("Zeit (s)")
plt.ylabel("Gyroskop y (rad/s)")
plt.legend()
plt.title("Dämpfung mit Peak-Detektion")
plt.grid()
plt.show()

# Plot: Logarithmus der Amplituden
plt.figure(figsize=(10, 5))
plt.plot(t_peaks, log_amplituden, "o", label="ln(Amplitude)")
plt.plot(t_peaks, linear(t_peaks, *params), "r--", label="Fit")
plt.xlabel("Zeit (s)")
plt.ylabel("ln(Amplitude)")
plt.title("Exponentielle Dämpfung")

```

```
plt.text(0.5, -0.05, "Abbildung 1: Beispielhafte Abbildungsunterschrift", ha="center", fontstyle="italic", fontweight="bold", fontfamily="serif", fontsize=10)
plt.legend()
plt.grid()
plt.show()
```

Dämpfungskonstante δ : 0.0039 1/s



5 5. Bestimmung der Komponenten der Beschleunigung

Um die einzelnen Komponenten der Beschleunigung zu berechnen, ist es wichtig zu wissen wo sich das Gyroskop genau befindet. Für den Versuch wurde ein Samsung S24 verwendet, doch da wir im Internet keinen genauen Bauplan des Geräts finden konnten, kann die genau Position nicht abgeschätzt werden. Deshalb nehmen wir für die Berechnung an, dass sich das Gyroskop genau mittig im Smartphone befindet.

Zum berechnen der Beschleunigung nutzen wir die gegebene Formel aus der Aufgabe, zum einen mit $\vec{g}$:

$$\vec{a} = \vec{g} + \vec{a_{zf}} + \vec{a_{\alpha}} = \vec{g} - \vec{\omega} \times (\vec{\omega} \times \vec{r}) - \vec{\alpha} \times \vec{r}$$

und ohne $\vec{g}$:

$$\vec{a} = \vec{a_{zf}} + \vec{a_{\alpha}} = -\vec{\omega} \times (\vec{\omega} \times \vec{r}) - \vec{\alpha} \times \vec{r}$$

Daraus folgt, dass wir nun die Vektoren für $\vec{r}$, $\vec{\omega}$ und $\vec{\alpha}$ aufstellen müssen um die Berechnung durchzuführen. Das Smartphone schwingt mit einem Abstand von $l = 0.93cm$ und dieses Seil verläuft entlang der negativen Z-Achse. Daraus folgt:

$$\vec{r} = \begin{pmatrix} 0 \\ 0 \\ -l \end{pmatrix}$$

Bei der Schwinung dreht sich das Smartphone um die Y-Achse, weshalb für $\vec{\omega}$ und $\vec{\alpha}$ gilt:

$$\vec{\alpha} = \begin{pmatrix} 0 \\ \alpha \\ 0 \end{pmatrix} \quad \vec{\omega} = \begin{pmatrix} 0 \\ \omega \\ 0 \end{pmatrix}$$

Für die Gravitation $\vec{g}$ erhält man:

$$\vec{g} = \begin{pmatrix} g \cdot \sin(\varphi) \\ 0 \\ -g \cdot \cos(\varphi) \end{pmatrix}$$

Wenn man nun alles in die Formel für die Beschleunigung mit g ($\vec{a}_g$) und ohne g ($\vec{a}$) einsetzt erhält man:

$$\vec{a}_g = \begin{pmatrix} 0 \\ -g \cdot \cos(\varphi) - l \cdot \omega^2 \end{pmatrix} \begin{pmatrix} l \cdot \alpha \\ 0 \\ -l \cdot \omega^2 \end{pmatrix}$$

Jetzt fehlen nur noch die Formeln für φ , ω und α . Da wir φ aus dem Aufgabenblatt entnehmen, können wir die anderen beiden jeweils durch zeitliche Ableitung von φ bilden.

$$\varphi = \varphi_0 \cos(\omega_0 t + \beta) \quad \omega = \dot{\varphi} = -\varphi_0 f_0 2\pi \sin(f_0 2\pi t) \quad \alpha = \ddot{\varphi} = -\varphi_0 f_0^2 4\pi^2 \cos(f_0 2\pi t)$$

Als erstes berechnen wir die Beschleunigung ohne g und erstellen jeweils einen Plot um die gemessenen Geschwindigkeitskomponenten jeder Achse mit den theoretischen zu vergleichen. Die Beschleunigung ohne g ist einfach die lineare Beschleunigung wie sie oben gezeigt wurde.

Die Pendellänge ist wie oben beschrieben $l = 0.93\text{cm}$, die Frequenz hatten wir auch oben bestimmt mit $f = 0.5169\text{Hz}$ und die Anfangsauslenkung unseres Pendels lag bei ca. 15° .

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# CSV einlesen
df = pd.read_csv('data/Linear Acceleration.csv')

# Filtere Daten im Bereich 20s bis 50s
df_filtered = df[(df['Time (s)'] >= 20) & (df['Time (s)'] <= 50)]

frequency = 0.5169
omega = 2 * np.pi * frequency
phi0 = 0.2618
l = 0.93
delta = 0.0039

# Neue Zeitachse für theoretische Kurve im Bereich 20-50s
time_theo = np.linspace(20, 50, 1000)
a_x_theo = l * phi0 * omega**2 * np.cos(omega * time_theo) * np.exp(-delta * time_theo)
a_y_theo = 0 * time_theo
```



```

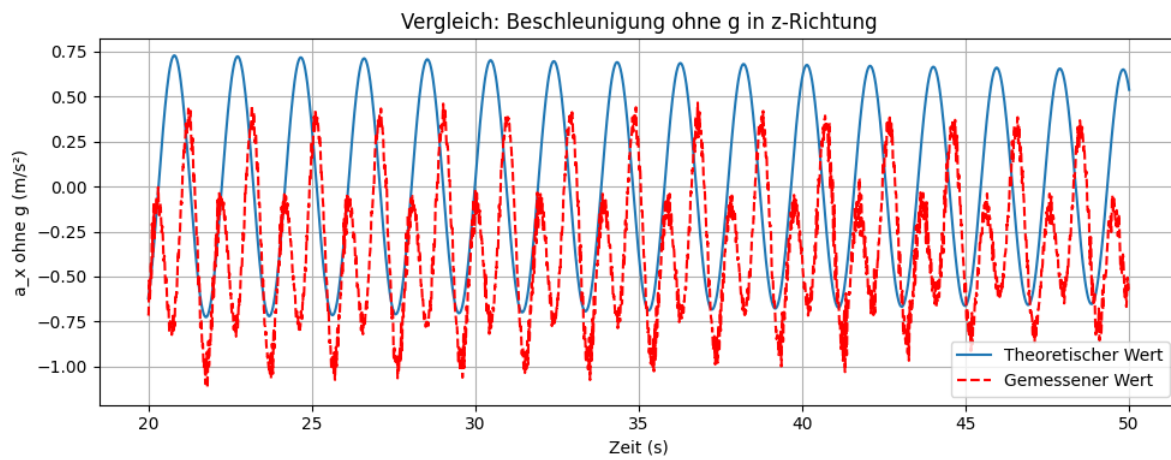
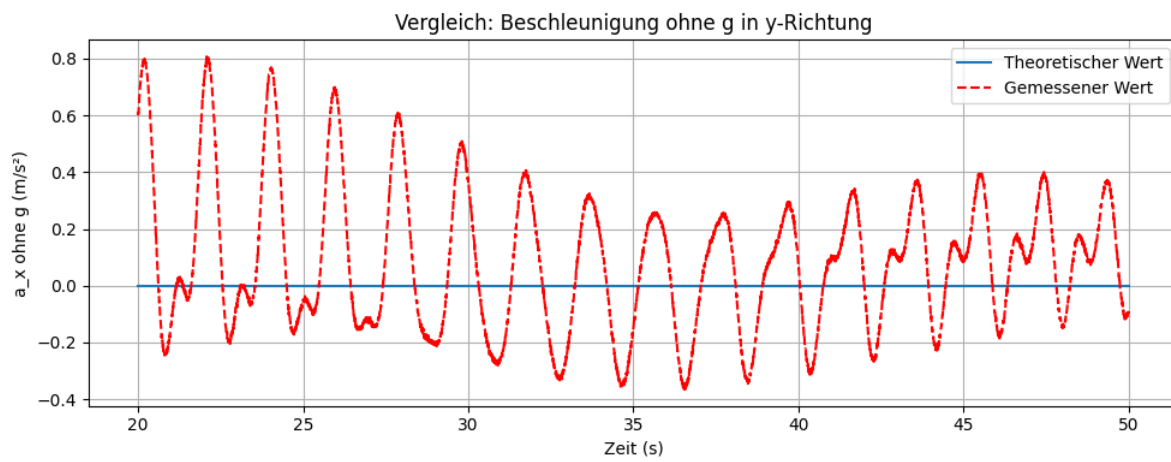
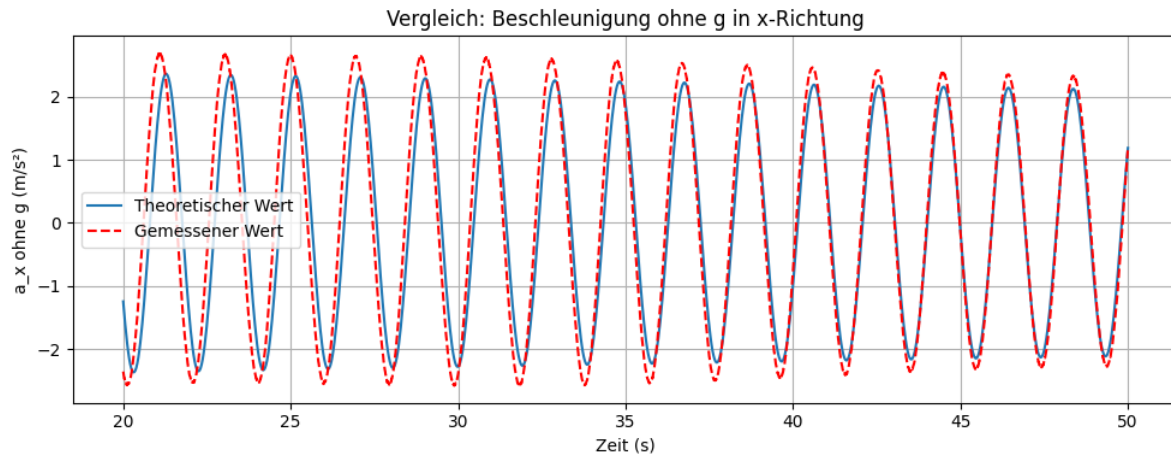
a_z_theo = - 1 * phi0 * frequency * 2 * np.pi * np.sin(frequency * 2 * np.pi * time_theo) * r

# Plot für a_x
plt.figure(figsize=(10, 4))
plt.plot(time_theo, a_x_theo, label="Theoretischer Wert")
plt.plot(df_filtered['Time (s)'], df_filtered['Linear Acceleration x (m/s^2)'], label="Gemessener Wert")
plt.xlabel("Zeit (s)")
plt.ylabel("a_x ohne g (m/s²)")
plt.title("Vergleich: Beschleunigung ohne g in x-Richtung")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

# Plot für a_y
plt.figure(figsize=(10, 4))
plt.plot(time_theo, a_y_theo, label="Theoretischer Wert")
plt.plot(df_filtered['Time (s)'], df_filtered['Linear Acceleration y (m/s^2)'], label="Gemessener Wert")
plt.xlabel("Zeit (s)")
plt.ylabel("a_y ohne g (m/s²)")
plt.title("Vergleich: Beschleunigung ohne g in y-Richtung")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

# Plot für a_z
plt.figure(figsize=(10, 4))
plt.plot(time_theo, a_z_theo, label="Theoretischer Wert")
plt.plot(df_filtered['Time (s)'], df_filtered['Linear Acceleration z (m/s^2)'], label="Gemessener Wert")
plt.xlabel("Zeit (s)")
plt.ylabel("a_z ohne g (m/s²)")
plt.title("Vergleich: Beschleunigung ohne g in z-Richtung")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

```



Jetzt betrachten wir noch die Beschleunigungskomponenten mit g.

```

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# CSV einlesen
df = pd.read_csv('data/Accelerometer.csv')

# Filtere Daten im Bereich 20s bis 50s
df_filtered = df[(df['Time (s)'] >= 20) & (df['Time (s)'] <= 50)]

# Neue Zeitachse für theoretische Kurve im Bereich 20-50s
time_theo = np.linspace(20, 50, 1000)

g = 9.81
frequency = 0.5169
omega = 2 * np.pi * frequency
phi0 = 0.2618
l = 0.93
phi = phi0 * np.cos(omega * time_theo)
delta = 0.0039

a_x_theo = (g * np.sin(phi) + l * -phi0 * omega**2 * np.cos(omega * time_theo)) * np.exp(-de
a_y_theo = 0 * time_theo
a_z_theo = -g * np.cos(phi) - l * (phi0 * frequency * 2 * np.pi * np.sin(frequency * 2 * np.]

# Plot für a_x
plt.figure(figsize=(10, 4))
plt.plot(time_theo, a_x_theo, label="Theoretischer Wert")
plt.plot(df_filtered['Time (s)'], df_filtered["Acceleration x (m/s^2)"], label="Gemessener W
plt.xlabel("Zeit (s)")
plt.ylabel("a_x mit g (m/s²)")
plt.title("Vergleich: Beschleunigung mit g in x-Richtung")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

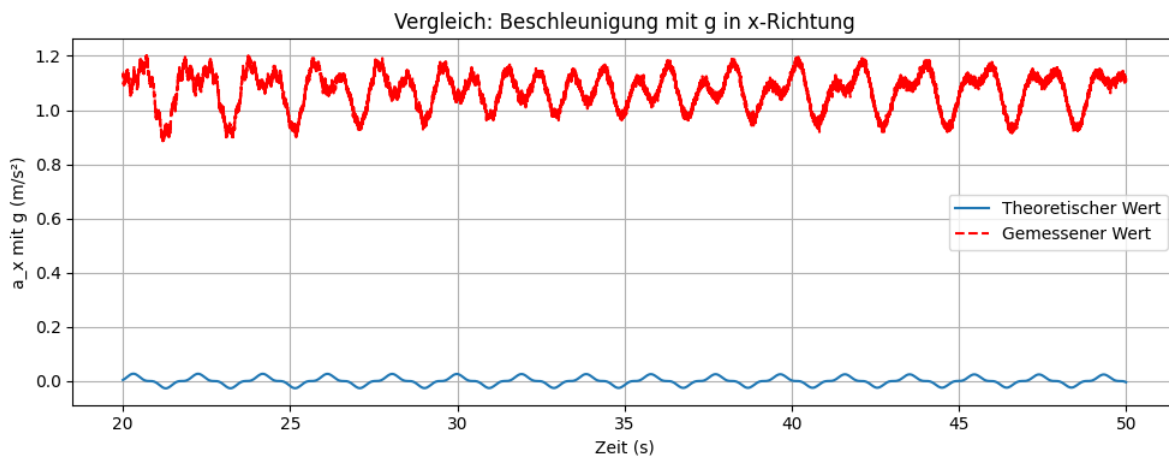
```

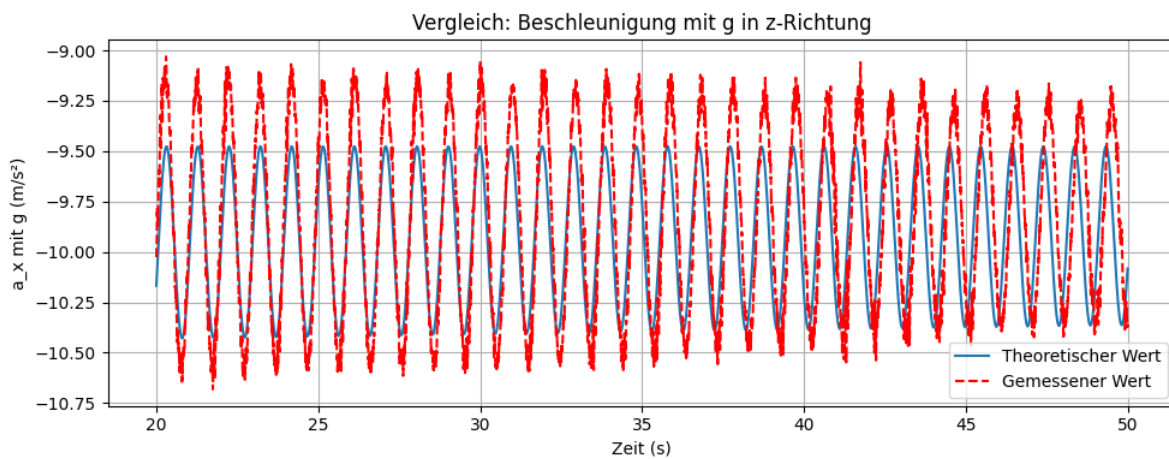
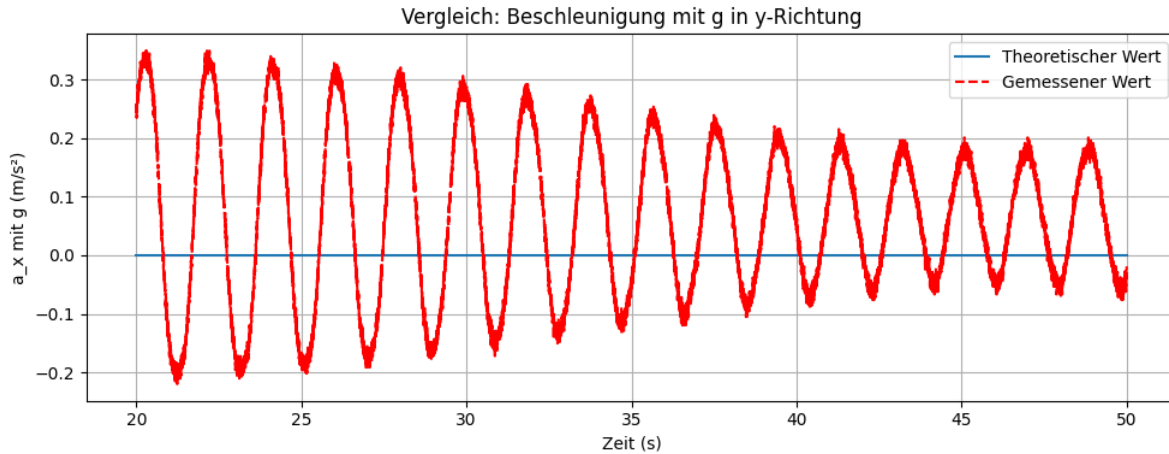
```

# Plot für a_y
plt.figure(figsize=(10, 4))
plt.plot(time_theo, a_y_theo, label="Theoretischer Wert")
plt.plot(df_filtered['Time (s)'], df_filtered["Acceleration y (m/s^2)"], label="Gemessener Wert")
plt.xlabel("Zeit (s)")
plt.ylabel("a_x mit g (m/s^2)")
plt.title("Vergleich: Beschleunigung mit g in y-Richtung")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

# Plot für a_z
plt.figure(figsize=(10, 4))
plt.plot(time_theo, a_z_theo, label="Theoretischer Wert")
plt.plot(df_filtered['Time (s)'], df_filtered["Acceleration z (m/s^2)"], label="Gemessener Wert")
plt.xlabel("Zeit (s)")
plt.ylabel("a_x mit g (m/s^2)")
plt.title("Vergleich: Beschleunigung mit g in z-Richtung")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

```





Nach den Berechnungen können wir sehen, dass viele der theoretischen und gemessenen Werte gut übereinstimmen. Vor allem bei der Beschleunigung in x-Richtung ohne g und der Beschleunigung in z-Richtung mit g gibt es große Ähnlichkeiten. Sehr große grafische Abweichungen gibt es bei der y-Achse, da in der Theorie dort keine Schwingung wäre, man dies in der Praxis aber nicht verhindern kann. Die gemessenen Werte sind somit nicht 0, allerdings sind die Werte im Vergleich zu den Achsen mit wirklicher Schwingung sehr viel kleiner. Große Abweichungen finden vor allem bei der x-Beschleunigung mit g statt. Da die Kurven selber nicht so voneinander abweichen, sondern die Kurve nur verschoben ist, scheint es bei der Messung eine falsche Zentrierung dieser Achse gegeben zu haben.

Die allgemeinen kleineren Unterschiede zwischen dem gemessenen und theoretischen Wert könnten durch verschiedene Ungenauigkeiten hervortreten. Zum einen wurde für die Formel von $\varphi(t)$ eine Kleinwinkelnäherung verwendet, welche nur sehr ähnlich zur Realität ist. Da wir keinen Aufbau des Smartphones im Internet finden konnten, haben wir auch angenommen, dass das Gyroskop mittig verbaut ist. Dies ist auch nur eine Abschätzung gewesen, welche

nicht der Realität gleichen muss. Desweiteren basieren unsere Berechnungen auf einem idealen Pendel mit einer Punktmasse, welche auch abweichend vom Smartphone ist.

Zusammenfassend lässt sich sagen, dass die theoretischen Werte häufig unseren Messungen geähnelt haben, doch durch verschiedene Faktoren, die in der Theorie ausgelassen werden, nicht genau die Schwingung des Smartphones hervorgesagt werden konnte.

6 6. Fehlerbetrachtungen

7 6. Fehlerbetrachtung

Für den gesamten Versuch gab es verschiedene Messungenauigkeiten, weshalb die Werte nicht genau der Theorie entsprechen.

- Wir wussten nicht genau wo sich das Gyroskop im Handy befindet, weshalb bei der Theorie die Annahme getroffen wurde, es befinde sich mittig. Dies kann zu größeren Unterschieden führen, wenn dies nicht der Fall ist.
- Bei Aufgabe 4 wurde für die Berechnung von $\varphi(t)$ eine Kleinwinkelnäherung verwendet. Diese kann in der Realität zu $\pm 1\%$ führen.
- Bei der Messung der Fadenlänge oder der Anfangsauslenkung kann es zu Messungenauigkeiten gekommen sein, da keine perfekten Messgeräte zur Verfügung standen.
- Das Handy ist keine perfekte Punktmasse und der Faden ist nicht masselos, obwohl diese Annahmen in der Theorie getroffen wurden.

8 7. Quellen

Alle Abbildungen wurden selber von den Autoren erstellt. Die Daten wurden mit der Phyphox App gemessen und die Diagramme wurden mit Python erstellt.